



Universidad Politécnica de Chiapas

Proyecto APrendIA

E S T A D Í A

Ingeniería en Software

Presenta:

Daniel Peregrino Perez

Asesor Laboral

Dr. Horacio Solis Cisneros

Asesor Académico

Dr. Carlos Alberto Díaz Hernández

Suchiapa, Chiapas, enero - abril de 2026

Dedicatoria

*A mis padres, por ser el pilar fundamental de mi vida,
por su amor incondicional y por creer en mí en cada etapa
de mi formación profesional.*

*Al Dr. Horacio Solís Cisneros, por su invaluable guía y paciencia
durante mi proceso de estadía. Gracias por compartir su conocimiento
en la limpieza de datos y por abrirme las puertas al fascinante mundo
de las Redes Neuronales, ResNet y YOLO; sus enseñanzas han sido
determinantes en el desarrollo de este proyecto.*

*Al Dr. Carlos Alberto Díaz Hernández, por brindarme las bases
sólidas en la materia de Inteligencia Artificial, despertando en mí
la curiosidad científica necesaria para afrontar este reto.*

*A mi tutor académico, Ali Santiago López, por su acompañamiento
constante y apoyo durante toda mi trayectoria académica en la
Universidad Politécnica de Chiapas.*

A todos ellos, dedico este trabajo como fruto del esfuerzo compartido.

Agradecimientos

Este trabajo es el resultado de años de esfuerzo, noches de estudio y, sobre todo, de la decisión de no rendirse a pesar de los retos que presenta la ingeniería. No hubiera sido posible llegar a esta meta sin el apoyo de personas que creyeron en mi capacidad incluso cuando yo mismo dudaba.

En primer lugar, agradezco profundamente a la **Universidad Politécnica de Chiapas** por ser mi casa de estudios y brindarme las herramientas para transformarme en profesional.

Agradezco de manera especial al **Dr. Horacio Solís Cisneros**. Gracias por su paciencia y por guiarme con tanta claridad en temas complejos como la limpieza de datos y la implementación de Redes Neuronales, ResNet y YOLO. Su mentoría fue la pieza clave que me permitió llevar este proyecto de una idea a una realidad funcional.

Al **Dr. Carlos Alberto Díaz Hernández**, por haberme brindado las bases fundamentales en Inteligencia Artificial. Sus clases no solo me dieron conocimiento técnico, sino que despertaron en mí la curiosidad por esta disciplina que hoy da vida a mi reporte de estadía.

A mi tutor académico, **Ali Santiago López**, por su acompañamiento constante, sus consejos y por estar pendiente de mi avance durante toda mi formación en la UPChiapas. Su apoyo administrativo y humano facilitó enormemente este proceso.

A mis padres, por su sacrificio y apoyo incondicional. Este título es tan suyo como mío, pues su confianza fue el motor que me impulsó a seguir adelante

en cada semestre.

Finalmente, a mis amigos y compañeros de clase, con quienes compartí el estrés de los exámenes y las alegrías de los proyectos logrados. Gracias por hacer que el camino fuera más ligero.

Resumen

El presente reporte técnico documenta el desarrollo e implementación de **Tutor_API**, una interfaz de programación de aplicaciones (API) robusta y asíncrona desarrollada con el framework **FastAPI** en lenguaje Python. El proyecto surge de la necesidad de integrar herramientas de inteligencia artificial en el programa de alfabetización “**Chiapas Puede**”, proporcionando retroalimentación automática e inmediata sobre la calidad de trazos manuscritos de niños y adultos en proceso de alfabetización.

La solución técnica integra dos modelos de *Deep Learning* exportados a formato ONNX para inferencia eficiente sin GPU: el detector **YOLOv8s**, que localiza caracteres manuscritos en fotografías reales de cuaderno, y el clasificador **EfficientNetV2-S + ArcFace**, capaz de reconocer 107 clases de caracteres del español —incluyendo vocales acentuadas, ñe y dígrafos— con un 79.34 % de exactitud sobre datos manuscritos reales.

Un componente crítico del desarrollo fue la fase de ingeniería de datos, donde se aplicaron técnicas rigurosas de **limpieza de datos** para asegurar el entrenamiento y funcionamiento óptimo de las redes neuronales. así como la normalización para la evaluación de caracteres.

Los resultados obtenidos demuestran que la integración de redes neuronales profundas en una infraestructura de API permite crear herramientas educativas más inteligentes, escalables y capaces de responder a las demandas de la educación moderna. Este trabajo representa la culminación de un proceso de estadía técnica donde se aplicaron conocimientos de desarrollo de software, visión artificial y gestión de datos.

Palabras clave: API, FastAPI, Redes Neuronales, YOLO, ResNet, Visión Artificial, Ges-

ción Educativa, Limpieza de Datos.

Índice general

Dedicatoria	1
Agradecimientos	2
Resumen	4
1. Introducción	12
1.1. Antecedentes	12
1.2. Planteamiento del problema	13
1.3. Objetivo general y objetivos específicos	13
1.4. Justificación del proyecto	15
1.5. Limitaciones y alcances	15
1.6. Entregables esperados	17
1.7. Actividades	17
1.8. Cronograma de actividades	17
2. Marco referencial	24
2.1. Marco teórico	24
2.2. Estado de la tecnología	28
2.2.1. Google Lookout / Seeing AI (Microsoft)	28
2.2.2. Duolingo (Sistema de aprendizaje adaptativo)	28
2.2.3. Sistemas de Reconocimiento de Escritura Manual (HTR)	29
2.2.4. Métricas de aprendizaje con margen angular	30
2.2.5. Frameworks de alfabetización digital en zonas rurales	31

2.3.	Contexto tecnológico	32
2.3.1.	Tecnologías utilizadas	32
2.3.2.	Pipeline de clasificación de caracteres	34
2.3.3.	Estrategia de entrenamiento	35
2.3.4.	Marco metodológico	36
2.3.5.	Tendencias actuales	36
2.3.6.	Comparativa de tecnologías	37
3.	Metodología y desarrollo del proyecto	38
3.1.	Levantamiento de requerimientos	38
3.2.	Análisis de requerimientos	39
3.3.	Diseño	40
3.4.	Implementación	42
3.4.1.	Carga de modelos al iniciar el servidor	42
3.4.2.	Endpoint de evaluación de carácter individual	42
3.4.3.	Pipeline de limpieza de imagen	43
3.4.4.	Entrenamiento de modelos en Kaggle	44
3.5.	Pruebas	45
3.6.	Despliegue	46
4.	Resultados	48
4.1.	Resultados mapeados a objetivos específicos	48
4.1.1.	Objetivo 1 — Infraestructura de <i>backend</i> (FastAPI)	48
4.1.2.	Objetivo 2 — Detector de caracteres YOLOv8s	48
4.1.3.	Objetivo 3 — Clasificador OCR (EfficientNetV2-S + ArcFace)	50
4.1.4.	Objetivo 4 — Pipeline de evaluación de trazo	51
4.1.5.	Objetivo 5 — Pipeline robusto de procesamiento de imagen	52
4.1.6.	Objetivo 6 — Cliente de escritorio Kivy	52
4.2.	Cumplimiento de entregables	53

5. Conclusiones y recomendaciones	54
5.1. Conclusiones	54
5.2. Recomendaciones	55
6. Anexos	58
6.1. Tarjeta de contacto de la empresa	58
6.2. Tarjeta de contacto del alumno	58
6.3. Carta de presentación	59
6.4. Carta de aceptación	60
6.5. Carta de terminación	61
6.6. Evaluación laboral	62
6.7. Evaluación docente	63
6.8. Repositorio y artefactos del proyecto	64
6.9. Glosario de abreviaturas	64

Índice de figuras

1.1. Diagrama de Gantt del proyecto Tutor_API.	18
1.2. Diagrama de Gantt del proyecto Tutor_API.	20
1.3. Diagrama de Gantt del proyecto Tutor_API.	21
1.4. Diagrama de Gantt del proyecto Tutor_API.	22
1.5. Diagrama de Gantt del proyecto Tutor_API.	23
3.1. Diagrama de componentes de Tutor_API: relación entre endpoints, orques- tador, pipelines de imagen y módulos de métricas.	41
4.1. Prototipo funcional del cliente de escritorio Kivy conectado a Tutor_API.	53

Índice de bloques de código

3.1. Carga única de modelos al iniciar el servidor FastAPI	42
3.2. Endpoint principal de evaluación de trazo (/evaluate)	42
3.3. Limpieza de líneas de cuaderno por inpainting	43
3.4. Comandos de despliegue del servidor	46
3.5. Despliegue con Docker	46

Índice de tablas

1.1. Lista de entregables con fecha de finalización.	17
1.2. Lista de actividades con fecha de finalización.	17
4.1. Endpoints implementados y verificados en Tutor_API.	49
4.2. Métricas del detector YOLOv8s en el conjunto de prueba.	49
4.3. Métricas del clasificador OCR v5 en el conjunto de prueba.	50
4.4. Composición del dataset del clasificador OCR.	50
4.5. Módulos de evaluación de trazo, pesos y descripción.	51
4.6. Tolerancias del módulo dt_coverage por nivel.	52
4.7. Desafíos del procesamiento de imagen y soluciones implementadas. . . .	52
4.8. Evidencia de cumplimiento de entregables.	53
6.1. Artefactos técnicos del proyecto Tutor_API.	65

Capítulo 1

Introducción

En el estado de Chiapas, el rezago educativo y el analfabetismo representan uno de los desafíos estructurales más significativos para el desarrollo social y económico. El programa **“Chiapas Puede”** surge como una iniciativa estatal para combatir esta problemática, buscando alfabetizar a niños y adultos en situación de vulnerabilidad. El presente proyecto, denominado **Proyecto APRENDIA**, se alinea con esta visión, proponiendo una herramienta tecnológica avanzada que utiliza Inteligencia Artificial para asistir en el proceso de enseñanza-aprendizaje de escritura de manera automatizada y con retroalimentación inmediata.

1.1. Antecedentes

El analfabetismo en Chiapas no es únicamente la incapacidad de leer o escribir, sino una barrera que limita el acceso a la información y a mejores oportunidades de vida. Históricamente, los esfuerzos de alfabetización han dependido de la disponibilidad física de tutores, lo cual es difícil de cubrir en las regiones más apartadas del estado.

Con el auge de la cuarta revolución industrial, ha surgido la oportunidad de utilizar la visión por computadora para crear “tutores inteligentes”. En este contexto, el proyecto se enmarca en la necesidad de modernizar las herramientas del programa “Chiapas Puede”, transitando de métodos tradicionales hacia sistemas interactivos capaces de analizar fotografías de cuadernos escolares y clasificar caracteres manuscritos —incluidos los propios

del español, como las vocales acentuadas y la ñe— mediante modelos de *Deep Learning*, facilitando así una experiencia de aprendizaje más intuitiva y accesible.

1.2. Planteamiento del problema

A pesar de los esfuerzos institucionales, el seguimiento individualizado de cada estudiante —ya sea niño o adulto en proceso de alfabetización— es una tarea monumental para cualquier docente. Los sistemas actuales suelen ser manuales o carecen de la capacidad de interactuar con el entorno físico del alumno y retroalimentarlo de forma inmediata.

La problemática central reside en la ausencia de una infraestructura tecnológica (*backend*) robusta que integre modelos de reconocimiento visual de manera escalable. Concretamente: el reconocimiento de caracteres manuscritos en 107 clases —incluyendo letras acentuadas del español, dígrafos como *ch* y *ll*, dígitos y signos de puntuación— junto con la evaluación cuantitativa de la calidad del trazo, requiere un pipeline de procesamiento de imagen que soporte fotos reales tomadas con celular en condiciones variables de iluminación. Sin una API eficiente que orqueste estos procesos, las aplicaciones educativas para alfabetización no pueden procesar en tiempo real la información necesaria para guiar al alumno, lo que limita sustancialmente el impacto de las herramientas digitales en sectores donde la lectura no es el medio principal de comunicación.

1.3. Objetivo general y objetivos específicos

El objetivo general del proyecto es *desarrollar una API robusta y escalable mediante el framework FastAPI que integre modelos de visión artificial —YOLOv8 para detección de caracteres manuscritos y un clasificador OCR basado en EfficientNetV2-S con cabeza de proyección y pérdida ArcFace para el reconocimiento de 107 clases de caracteres— sirviendo como motor de un sistema de tutoría inteligente de caligrafía enfocado en el programa de alfabetización “Chiapas Puede”*.

Los objetivos específicos del proyecto son:

- Diseñar e implementar una arquitectura de *backend* asíncrona utilizando **FastAPI**

con tres endpoints principales: evaluación de carácter individual (`/evaluate`), evaluación de plana completa (`/evaluate_plana`) y reconocimiento de texto (`/recognize`), garantizando tiempos de respuesta de 200–500 ms en CPU.

- Integrar el modelo **YOLOv8s** como detector *single-class* de caracteres manuscritos (clase única: *trazo*), entrenado sobre un dataset compuesto de imágenes reales y composites sintéticos multi-carácter que simulan planas escolares con fondos variados.
- Diseñar y entrenar un **clasificador OCR de 107 clases** basado en **EfficientNetV2-S** con cabeza de proyección lineal (1280→512), pérdida **ArcFace** ($s=30$, $m=0.15$) y **Focal Loss** con pesos por clase, capaz de reconocer letras mayúsculas, minúsculas, dígitos, dígrafos y caracteres propios del español (á, é, í, ó, ú, ü, ñ y sus versiones en mayúscula).
- Desarrollar una estrategia de **aumento de datos por acentuación** (*accent augmentation*) que, a partir de imágenes reales del dataset EMNIST, genere 400 muestras por clase de carácter acentuado superponiendo el diacrítico correspondiente (tilde, diéresis, virgulilla) sobre la letra base, compensando así la ausencia de datos reales etiquetados para estas clases en datasets públicos.
- Implementar un **pipeline de evaluación de trazo** con 8 métricas especializadas — distance transform (precisión y cobertura), topología, SSIM, Procrustes, Hausdorff, DTW y similitud coseno— que califica la calidad del trazo manuscrito en una escala de 0 a 100 con tres niveles de tolerancia: principiante (8 px), intermedio (5 px) y avanzado (3 px).
- Diseñar un **pipeline robusto de procesamiento de imagen** para fotografías reales de cuaderno, con eliminación automática de líneas azules por inpainting, normalización de iluminación adaptativa, binarización por percentiles y una doble red de seguridad con múltiples fallbacks que garantice la generación de máscaras válidas incluso en condiciones fotográficas adversas.

1.4. Justificación del proyecto

La implementación de **Tutor_API** es fundamental porque democratiza el acceso a tecnología de evaluación formativa para quienes más la necesitan. Al utilizar inteligencia artificial, el programa “Chiapas Puede” puede ofrecer una herramienta que no solo registra datos, sino que *evalúa* lo que el alumno escribe: analiza la fotografía de su cuaderno, detecta cada carácter manuscrito, lo reconoce y califica la calidad de su trazo comparándolo con una plantilla de referencia, generando retroalimentación pedagógica inmediata en español sin requerir la presencia física de un docente.

Desde el punto de vista técnico, el uso de **FastAPI** con servidor ASGI garantiza que el sistema sea ligero y capaz de funcionar incluso en servidores con recursos modestos, relevante para la infraestructura disponible en instituciones educativas de Chiapas. La arquitectura del clasificador OCR —basada en **EfficientNetV2-S** con *metric learning* mediante **ArcFace**— produce un modelo exportable a formato ONNX que permite inferencia en CPU sin necesidad de GPU en el servidor de producción, con latencias de 200–500 ms por carácter. Este proyecto no solo cumple con un requisito académico de estadía, sino que aporta una solución técnica real a una problemática social crítica del estado.

1.5. Limitaciones y alcances

Alcances:

- Desarrollo completo del *backend* con los tres endpoints de la API (/evaluate, /evaluate_plana, /recognize) y documentación interactiva generada automáticamente con Swagger.
- Integración funcional de **YOLOv8s** para detección de caracteres manuscritos y del **clasificador OCR** (EfficientNetV2-S + ArcFace, 107 clases), ambos exportados a formato ONNX para inferencia en producción.
- Entrenamiento del clasificador OCR en Kaggle con un pipeline que incluye datos

reales de EMNIST y verack, generación sintética con fuentes tipográficas manuscritas, y aumento por acentuación (*accent augmentation*), alcanzando un 79.34 % de exactitud sobre datos reales en el conjunto de prueba.

- Pipeline de evaluación de trazo con 8 métricas especializadas, 3 niveles de dificultad y generación de retroalimentación pedagógica en español.
- Pipeline robusto de procesamiento de imagen para fotos reales de cuaderno, con eliminación de líneas azules por inpainting y doble red de seguridad con fallbacks automáticos.
- Cliente de escritorio funcional desarrollado con **Kivy 2.3.0** que se comunica con el *backend* vía HTTP para uso directo por el alumno.
- Inclusión de un **Dockerfile** en el repositorio para facilitar el despliegue en contenedores.

Limitaciones:

- El proyecto se centra en el *backend* (API) y el cliente de escritorio; el desarrollo de una aplicación móvil nativa (iOS/Android) no forma parte del alcance de este reporte.
- El sistema no reconoce escritura cursiva continua; trabaja exclusivamente con caracteres manuscritos separados entre sí.
- La precisión del clasificador en datos reales es de aproximadamente 79.34 %, lo que implica un margen de error de ≈ 20 % en condiciones de producción. Las clases de caracteres acentuados se apoyan en *accent augmentation* al no existir datasets públicos con muestras reales suficientes de á, é, í, ó, ú, ü y ñ manuscritas.
- El sistema requiere conexión a internet o red local para interactuar con la API, lo cual puede ser una restricción en zonas de nula cobertura.
- No se implementó persistencia en base de datos; cada evaluación es independiente y el sistema no registra el progreso histórico del alumno en esta versión.

1.6. Entregables esperados

Los entregables comprenden el código fuente del servidor, los modelos entrenados y su documentación de uso. La lista de entregables se presenta en la Tabla 1.1.

Tabla 1.1: Lista de entregables con fecha de finalización.

ID	Entregable	Fe
1	Repositorio de código (Tutor_API)	
2	Documentación de endpoints (Swagger / /docs)	
3	Modelos entrenados y exportados a ONNX (detector YOLOv8s + clasificador EfficientNetV2-S)	
4	Reporte final de estadía	

1.7. Actividades

La Tabla 1.2 resume las actividades desarrolladas durante el periodo de estadía.

Tabla 1.2: Lista de actividades con fecha de finalización.

ID	Actividad
1	Análisis de requerimientos del programa “Chiapas Puede”
2	Recopilación, limpieza y aumento del dataset de caracteres (EMNIST, verack, sintéticos, <i>accent aug</i>
3	Entrenamiento del detector YOLOv8s y del clasificador EfficientNetV2-S + ArcFace en Kaggle; expo
4	Desarrollo de la API en FastAPI: endpoints, pipeline de imagen, métricas de trazo y generación de p
5	Integración del cliente Kivy, pruebas con datos reales y documentación final

1.8. Cronograma de actividades

El cronograma fue diseñado para cubrir las 15 semanas de estadía técnica, priorizando primero el entendimiento de los datos y el entrenamiento de modelos, y culminando con la integración completa de la API y las pruebas con datos reales de cuaderno.

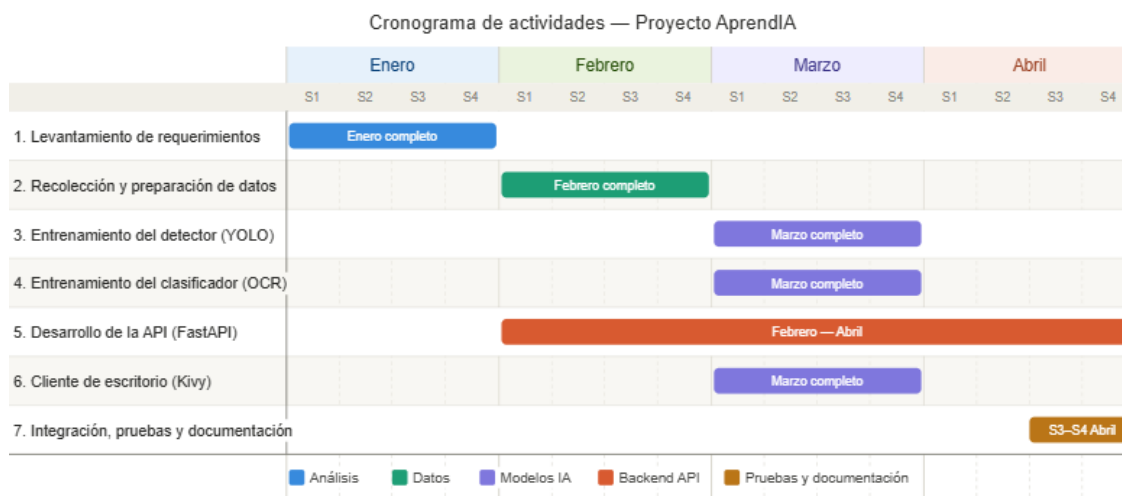


Figura 1.1: Diagrama de Gantt del proyecto Tutor_API.

Para validar el cumplimiento de las actividades en los tiempos establecidos en el cronograma, se presenta a continuación la evidencia del trabajo técnico realizado en las distintas fases del proyecto:

Evidencia de desarrollo y sesiones de trabajo

A lo largo de la estadía, se mantuvieron sesiones de desarrollo documentadas que avalan el progreso técnico:

- **Fase de Análisis y Procesamiento:** Desarrollo de lógica base - Ver registro.
- **Diseño de Arquitectura API:** Estructura de endpoints y servicios - Ver registro.
- **Implementación de Modelos:** Optimización de respuestas y lógica - Ver registro.
- **Refinamiento y Depuración:** Corrección de errores y pruebas de flujo - Ver registro.
- **Integración Final:** Validación de procesamiento de datos - Ver registro.
- **Pruebas de Despliegue:** Configuración final y documentación de la API - Ver registro.

Continuidad en plataforma Kaggle

Como respaldo adicional de la carga de trabajo y el proceso iterativo, el desarrollo del núcleo de procesamiento se llevó a cabo en la plataforma Kaggle. Se registran un total de **53 versiones** del proyecto, lo que evidencia un proceso de mejora continua y pruebas rigurosas.

Cabe destacar que las versiones finales (como la versión 53) requirieron tiempos de ejecución de más de **2 horas por sesión**, lo que demuestra la complejidad técnica y el volumen de datos procesados durante el tiempo asignado en el cronograma de actividades.

Viewing Version 53: ✓✓

← Save & Run All • April 14, 2026 at 11:41 AM

 Edit

Version History



Version 8

1mo ago

Save & Run All • Diff: +26 -42

Failed after 1 minute and 55 seconds

...



Version 7

1mo ago

Save & Run All • Diff: +76 -43

Failed after 18 minutes and 53 seconds

...



Version 6

1mo ago

Save & Run All • Diff: +26 -9

Failed after 56 minutes and 22 seconds

...



Version 5

1mo ago

Save & Run All • Diff: +17 -18

Failed after 2 minutes and 7 seconds

...



Version 4

1mo ago

Save & Run All • Diff: +0 -0

Failed after 25 seconds

...



Version 3

1mo ago

Save & Run All • Diff: +0 -0

Cancelled after 0 seconds

...

13:01



Vo
LTE



kaggle.com/code/danielc



Viewing Version 53: ✓✓



Save & Run All • April 14,
2026 at 11:41 AM



Edit

Version History



Version 23

23d ago

Save & Run All • Diff: +3868 -223

Cancelled after 12 hours



Version 22

25d ago

Save & Run All • Diff: +88 -55

Cancelled after 12 hours



Final Final

1mo ago

Save & Run All • Diff: +74 -2

Failed after 10 hours and 44 minutes



Final Hope

1mo ago

Save & Run All • Diff: +136 -130

Failed after 14 minutes and 22 seconds



Version 19

1mo ago

Save & Run All • Diff: +18 -20

Failed after 4 hours and 21 minutes



13:01



Vo
LTE



kaggle.com/code/danielc



Viewing Version 53: ✓✓



Save & Run All • April 14,



2026 at 11:41 AM



Edit

Version History



Version 42

12d ago

Save & Run All • Diff: +95 -52

Ran in 1 hour and 41 minutes



Version 41

13d ago

Save & Run All • Diff: +94 -10

Failed after 1 hour and 36 minutes



Version 40

13d ago

Save & Run All • Diff: +594 -427

Failed after 2 hours and 6 minutes



LAST

14d ago

Save & Run All • Diff: +453 -125

Failed after 1 minute and 49 seconds



FINAL FINAL

14d ago

Save & Run All • Diff: +771 -978

Failed after 1 hour and 19 minutes



Version History



Version 53

Save & Run All • Diff: +17 -36

Ran in 2 hours and 7 minutes

7d ago





Version 52

Save & Run All • Diff: +8 -2

Failed after 2 hours and 2 minutes

7d ago





Version 51

Save & Run All • Diff: +12 -3

Failed after 2 hours and 3 minutes

8d ago





Ultima version

Save & Run All • Diff: +3290 -674

Failed after 7 minutes and 44 seconds

8d ago





Version 49

Save & Run All • Diff: +354 -0

Ran in 1 hour and 44 minutes

10d ago





Version 48

Save & Run All • Diff: +12 -0

Failed after 1 hour and 44 minutes

10d ago



Capítulo 2

Marco referencial

El presente capítulo establece los fundamentos teóricos y el panorama tecnológico que sustentan el desarrollo de **Tutor_API**. Se analizan los conceptos de visión artificial y desarrollo de servicios web, así como el estado actual de las herramientas de apoyo a la alfabetización, permitiendo justificar la selección de tecnologías frente a las necesidades del programa “Chiapas Puede”.

2.1. Marco teórico

- **Contextualización del proyecto:** La alfabetización es un proceso complejo que requiere una interacción constante entre tutor y aprendiz. En entornos rurales o con escasez de docentes, la tecnología debe actuar como un puente. Desde un enfoque conceptual, este proyecto aborda la brecha educativa mediante la *asistencia inteligente*, donde el software no solo presenta información, sino que “percibe” el cuaderno del estudiante a través de fotografías para detectar caracteres manuscritos, reconocerlos y calificar la calidad del trazo de forma autónoma.
- **Definición de conceptos clave:**
 - **API (Application Programming Interface):** Interfaz que permite la comunicación entre dos aplicaciones mediante contratos bien definidos de solicitud y respuesta. En este proyecto actúa como el motor centralizado que recibe

imágenes de cuaderno y devuelve puntuaciones de trazo, caracteres reconocidos y retroalimentación pedagógica en español.

- **FastAPI:** Framework de Python de alto rendimiento basado en *Starlette* y *Pydantic* que maneja peticiones asíncronas (*async/await*), genera documentación interactiva (Swagger/OpenAPI) de forma automática y valida los tipos de datos de entrada y salida, lo que lo hace idóneo para exponer modelos de IA pesados sin bloquear el servidor.
- **Visión por computadora (Computer Vision):** Rama de la inteligencia artificial que entrena a las computadoras para interpretar y comprender el contenido visual de imágenes o vídeos. En este proyecto se aplica para localizar caracteres manuscritos en fotografías de cuadernos escolares y extraer representaciones que permitan reconocerlos y evaluar su forma.
- **YOLO (You Only Look Once):** Familia de modelos de detección de objetos en tiempo real que divide la imagen en una cuadrícula y predice simultáneamente las coordenadas de los cuadros delimitadores (*bounding boxes*) y las probabilidades de clase en un único paso de inferencia. En este proyecto se emplea **YOLOv8s** como detector *single-class* de caracteres manuscritos: la única clase detectada es *trazo*, es decir, cualquier carácter escrito a mano, independientemente de cuál sea.
- **EfficientNetV2-S:** Variante compacta de la familia EfficientNetV2, diseñada por Google, que optimiza simultáneamente la precisión y la velocidad de entrenamiento mediante un escalado compuesto mejorado y operaciones *Fused-MBConv*. En este proyecto se emplea como extractor de características visuales (*backbone*) preentrenado en ImageNet, produciendo vectores de 1 280 dimensiones mediante *global average pooling*.
- **ArcFace (Additive Angular Margin Loss):** Función de pérdida propuesta por Deng et al. (2019) que proyecta las representaciones aprendidas (*embeddings*) sobre una hiperesfera unitaria y añade un margen angular aditivo m a la clase correcta antes de aplicar *softmax*, forzando al modelo a separar clases

visualmente similares —como a/á o c/C— con mayor discriminación que una función *softmax* estándar. La función de pérdida se define como:

$$\mathcal{L} = -\log \frac{e^{s(\cos(\theta_{y_i} + m))}}{e^{s(\cos(\theta_{y_i} + m))} + \sum_{j \neq y_i} e^{s \cos \theta_j}}$$

donde θ_{y_i} es el ángulo entre el *embedding* y el prototipo de la clase correcta, s es el factor de escala y m es el margen angular. En este proyecto se emplean $s = 30$ y $m = 0.15$.

- **Projection Head:** Capa intermedia que reduce la dimensionalidad de las características del *backbone* (1 280 dimensiones) a un espacio más compacto (512 dimensiones) mediante una transformación lineal seguida de normalización por lotes (*Batch Normalization*), activación ReLU y *Dropout* (tasa 0.4). Permite al modelo aprender qué características son relevantes para distinguir pares confusos antes de la clasificación.
- **Focal Loss:** Variante de la entropía cruzada propuesta por Lin et al. (2017) que reduce dinámicamente el peso de los ejemplos bien clasificados (*fáciles*) y concentra el entrenamiento en los difíciles, mediante un factor modulador $(1 - p_t)^\gamma$. En este proyecto se usa $\gamma = 2.0$, combinada con pesos por clase calculados mediante el *Effective Number of Samples* (Cui et al., 2019) y un factor de impulso adicional de $1.5\times$ para las 14 clases de caracteres acentuados.
- **Transfer Learning:** Técnica que reutiliza un modelo preentrenado en una tarea general (clasificación de ImageNet) y lo adapta a una tarea específica (reconocimiento de caracteres manuscritos), reduciendo drásticamente los datos y el tiempo de entrenamiento necesarios.
- **Accent Augmentation:** Técnica de aumento de datos desarrollada en este proyecto para generar imágenes de caracteres acentuados del español (á, é, í, ó, ú, ü, ñ y sus mayúsculas) dibujando diacríticos —acentos agudos, diéresis y virgulillas— sobre imágenes manuscritas reales del dataset EMNIST. La técnica detecta la posición del carácter base, desplaza la imagen si es necesario para dejar espacio al diacrítico, y lo traza con variaciones aleatorias de posición,

tamaño y grosor, preservando la apariencia manuscrita natural. Se generan 400 imágenes por clase acentuada.

- **Data Augmentation:** Conjunto de transformaciones aplicadas a las imágenes de entrenamiento —rotaciones, deformaciones elásticas, operaciones morfológicas (erosión/dilatación para simular variaciones de grosor de trazo), variaciones de brillo y contraste, ruido gaussiano, *Coarse Dropout*— para incrementar artificialmente la diversidad del dataset y mejorar la generalización del modelo.
- **ONNX (Open Neural Network Exchange):** Formato estándar abierto para la representación de modelos de aprendizaje profundo, que permite entrenar en un framework (PyTorch) y desplegar en un entorno de ejecución optimizado (ONNX Runtime) con independencia de la plataforma y sin necesidad de que el servidor de producción tenga PyTorch instalado.
- **Limpieza de datos (Data Cleaning):** Proceso de detección y corrección de registros corruptos o imprecisos en un dataset, incluyendo deduplicación por hash MD5, saneamiento de coordenadas de *bounding boxes* y verificación de calidad de imagen, vital para que los modelos de IA no aprendan patrones incorrectos.
- **Distance Transform:** Operación de procesamiento de imagen que, para cada píxel de fondo en una imagen binaria, calcula su distancia al píxel de trazo más cercano. En el sistema de evaluación de trazo, se aplica sobre la plantilla de referencia y se evalúa en los puntos del esqueleto del alumno: valores bajos indican que el trazo está cerca del ideal.
- **SmartOCR:** Módulo de post-procesamiento desarrollado en este proyecto que combina las predicciones individuales del clasificador OCR con agrupación espacial por palabras, contexto posicional y corrección por diccionario con distancia de Levenshtein, produciendo texto reconocido más coherente que la clasificación carácter a carácter sin contexto.

2.2. Estado de la tecnología

A continuación se presentan cinco trabajos y herramientas que representan el estado del arte en tecnología educativa y visión artificial relevantes para este proyecto.

2.2.1. Google Lookout / Seeing AI (Microsoft)

Contexto

Aplicaciones diseñadas para personas con discapacidad visual que utilizan la cámara del móvil para identificar objetos, leer textos y describir escenas en tiempo real.

Tecnologías aplicadas

Modelos de *Deep Learning* optimizados para dispositivos móviles (*Edge Computing*) y servicios en la nube (Azure/Google Cloud).

Fortalezas y debilidades

Fortalezas: Gran precisión y accesibilidad. **Debilidades:** Dependencia total de servicios propietarios costosos y enfoque generalista, sin estructura pedagógica.

Diferenciador con Tutor_API

Estas aplicaciones carecen de un módulo de evaluación de calidad de trazo, retroalimentación pedagógica estructurada y soporte para los caracteres propios del español como las vocales acentuadas y la ñe, que son el núcleo del presente proyecto.

2.2.2. Duolingo (Sistema de aprendizaje adaptativo)

Contexto

Plataforma de aprendizaje de idiomas que implementa algoritmos adaptativos para personalizar el ritmo y el contenido de las lecciones según el desempeño del usuario.

Tecnologías aplicadas

Redes neuronales convolucionales integradas en aplicaciones móviles y modelos de espaciado de repetición (*spaced repetition*) para la retención.

Fortalezas y debilidades

Fortalezas: Excelente gamificación y experiencia de usuario. **Debilidades:** El contenido es rígido y no está diseñado para personas analfabetas funcionales que requieren guía desde el trazo individual; tampoco evalúa la calidad caligráfica de lo que el usuario escribe.

Diferenciador con Tutor_API

Tutor_API está diseñado específicamente para el contexto de alfabetización en español, incorporando detección de caracteres manuscritos en fotografías de cuadernos reales y evaluación cuantitativa del trazo con retroalimentación inmediata, capacidades que DuoLingo no implementa.

2.2.3. Sistemas de Reconocimiento de Escritura Manual (HTR)

Contexto

Diversas investigaciones universitarias y sistemas comerciales emplean redes neuronales profundas para reconocer caracteres manuscritos, con aplicaciones en la enseñanza de la escritura y la digitalización de documentos históricos. El dataset **EMNIST** (*Extended MNIST*), que contiene más de 800 000 muestras de caracteres manuscritos en 62 clases (dígitos, letras mayúsculas y minúsculas), se ha consolidado como referencia para el entrenamiento y la evaluación de estos sistemas. Sin embargo, EMNIST no incluye caracteres propios del español como las vocales acentuadas o la eñe, lo que representa un vacío significativo para aplicaciones de alfabetización en este idioma.

Tecnologías aplicadas

Arquitecturas basadas en CNN profundas —EfficientNet, ResNet— combinadas con funciones de pérdida con margen angular (ArcFace, CosFace) para mejorar la discriminación entre clases similares, y técnicas de aumento de datos con transformaciones elásticas y morfológicas.

Fortalezas y debilidades

Fortalezas: Muy precisos en tareas específicas; la combinación de *Transfer Learning* con pérdidas de margen angular permite distinguir clases sutilmente diferentes. **Debilidades:** La inexistencia de datos manuscritos reales para caracteres acentuados del español en conjuntos de datos públicos limita la cobertura de estos sistemas para el español.

Diferenciador con Tutor_API

Además de servir el modelo vía **FastAPI** con inferencia en **ONNX Runtime** (sin necesidad de GPU en el servidor), este proyecto implementa una técnica original de **Accent Augmentation** que genera datos realistas de los 14 caracteres acentuados del español dibujando diacríticos sobre imágenes manuscritas reales de EMNIST, eliminando la dependencia de datos etiquetados inexistentes para estas clases.

2.2.4. Métricas de aprendizaje con margen angular

Contexto

La familia de funciones de pérdida con margen angular —ArcFace (Deng et al., 2019), CosFace, SphereFace— fue desarrollada para reconocimiento facial, donde el reto es distinguir rostros similares en un espacio de alta dimensionalidad. Recientemente estas técnicas se han trasladado a dominios como reconocimiento de caracteres, verificación de firmas y reconocimiento de productos.

Tecnologías aplicadas

Redes convolucionales profundas con capas de proyección y normalización L2, combinadas con funciones de pérdida que imponen un margen angular en el espacio de *embeddings*.

Fortalezas y debilidades

Fortalezas: Excelente discriminación entre clases visualmente similares; aprendizaje de representaciones compactas y generalizables. **Debilidades:** Requiere estabilización cuidadosa del entrenamiento (*warmup*, congelación progresiva del *backbone*) y es sensible a los hiperparámetros de escala s y margen m .

Diferenciador con Tutor API

Este proyecto adapta **ArcFace** al dominio específico de los 107 caracteres del español, incorporando un *Projection Head* ($1280 \rightarrow 512$), *Focal Loss* con pesos por clase para gestionar el desbalance natural entre clases con datos reales y clases sintéticas, y un esquema de descongelamiento progresivo del *backbone* con *warmup* lineal para estabilizar el entrenamiento.

2.2.5. Frameworks de alfabetización digital en zonas rurales

Contexto

Programas gubernamentales que distribuyen tabletas con contenido educativo precargado en zonas de baja conectividad.

Tecnologías aplicadas

Aplicaciones nativas Android sin conexión a internet, con contenido audiovisual estático.

Fortalezas y debilidades

Fortalezas: No requieren internet. **Debilidades:** No tienen capacidad de procesamiento de IA; solo presentan videos o imágenes estáticas sin retroalimentación sobre lo que el alumno produce.

Diferenciador con Tutor_API

La inclusión de IA permite una interacción dinámica: el sistema analiza la fotografía del cuaderno del alumno, detecta cada carácter manuscrito y califica la calidad de su trazo comparándolo con una plantilla de referencia, proporcionando retroalimentación inmediata mucho más rica que un video estático. La exportación del modelo a **ONNX** abre además la posibilidad futura de ejecutar la inferencia directamente en el dispositivo sin conexión.

2.3. Contexto tecnológico

2.3.1. Tecnologías utilizadas

- **Lenguaje:** Python 3.10.11, elegido por su ecosistema de librerías de IA y su amplia adopción en investigación y producción.
- **Framework backend:** **FastAPI** con servidor ASGI **Uvicorn**, por su capacidad de manejar peticiones asíncronas y la autogeneración de documentación interactiva (Swagger/OpenAPI) disponible en /docs.
- **IA y visión — Detección de caracteres:** Modelo **YOLOv8s** entrenado como detector *single-class* (clase 0 = *trazo*) que localiza todos los caracteres manuscritos presentes en la imagen, independientemente de cuáles sean, devolviendo sus coordenadas de *bounding box* para recortarlos y pasarlos al clasificador.
- **IA y visión — Clasificación de caracteres:**
 - **Backbone:** **EfficientNetV2-S** (tf_efficientnetv2_s, biblioteca *timm* 0.9.16)

preentrenado en ImageNet, produciendo vectores de 1 280 dimensiones mediante *global average pooling*.

- **Projection Head:** $\text{Linear}(1\,280 \rightarrow 512) + \text{BatchNorm} + \text{ReLU} + \text{Dropout}(0.4)$, que comprime las representaciones a un espacio de 512 dimensiones optimizado para similitud coseno.
- **Cabeza de clasificación: ArcFace** ($s = 30$, $m = 0.15$) que separa las 107 clases en un espacio hipersférico.
- **Función de pérdida: Focal Loss** ($\gamma = 2.0$) con suavizado de etiquetas ($\varepsilon = 0.05$) y pesos por clase calculados mediante *Effective Number of Samples* (Cui et al., 2019), con factor de impulso $1.5\times$ para las 14 clases de caracteres acentuados.

■ **Datos de entrenamiento:**

- **EMNIST ByClass:** Hasta 800 imágenes por clase, cubriendo dígitos y letras latinas sin acentos (62 clases). Las imágenes se corrigen de orientación (*transpose + flip + inversión de color*) antes de usarse.
- **Verack Spanish Handwritten:** Miles de imágenes de caracteres manuscritos en español (incluyendo letras acentuadas y signos de puntuación), accedidas mediante un escaneo con mapeo NFC-normalizado de nombres de carpeta a carácter Unicode.
- **Accent Augmentation:** $\sim 5\,600$ imágenes generadas dibujando diacríticos sobre imágenes manuscritas reales de EMNIST (400 por cada una de las 14 clases acentuadas).
- **Generación sintética:** Imágenes tipográficas generadas con fuentes manuscritas del sistema, deformaciones elásticas ($\alpha = 1.5$, $p = 0.4$), variaciones morfológicas y ruido, cubriendo las 31 clases sin representación real suficiente (signos de puntuación, dígrafos, trazos básicos).

- **Partición honesta de datos:** Los conjuntos de validación y prueba contienen **exclusivamente** imágenes de fuentes reales (EMNIST, Verack) y de *Accent Augmen-*

tation, mientras que las imágenes puramente sintéticas se destinan en su totalidad al entrenamiento. Esto garantiza que las métricas reflejen el rendimiento ante escritura manuscrita real: 79.34 % de exactitud en datos reales, frente al 97.62 % inflado que arrojaría evaluar sobre datos sintéticos.

- **Aumento de datos en entrenamiento:** Biblioteca **Albumentations** (v1.3.1) con rotación (± 12), perspectiva, distorsión elástica ($\alpha = 1.5$), operaciones morfológicas, ruido gaussiano, *Coarse Dropout* y *Mixup* ($\alpha = 0.2$). Se emplea un `WeightedRandomSampler` con pesos diferenciados por fuente: Verack 1.5×, *Accent Augmentation* 1.3×, sintéticos 0.7×.
- **Exportación e inferencia:** Modelo exportado a **ONNX** (opset 17), incluyendo el *backbone*, el *Projection Head* y los pesos de ArcFace, para inferencia con **ONNX Runtime** (v1.18.1) con soporte para *Test-Time Augmentation* (TTA, 5 transformaciones).
- **Framework de entrenamiento:** **PyTorch** (v2.2.0) con precisión mixta (`torch.cuda.amp`), biblioteca **timm** para modelos preentrenados, y **scikit-learn** para métricas de evaluación. El entrenamiento se realizó en **Kaggle Notebooks** con GPU T4 (16 GB VRAM).
- **Librerías de apoyo:** **Pydantic** para validación de datos en la API; **OpenCV** (v4.9.0.80, *headless*) para preprocesamiento de imagen (redimensionamiento *letterbox*, inpainting de líneas azules, binarización, operaciones morfológicas); **scikit-image** para esqueletización de trazos; **SciPy** para alineamiento por *Dynamic Time Warping* (DTW); **PIL/Pillow** para generación de imágenes sintéticas; y **MLflow** (v2.10.2) para registro de experimentos.

2.3.2. Pipeline de clasificación de caracteres

El flujo completo de procesamiento para la clasificación de un carácter manuscrito se describe a continuación:

1. **Preprocesamiento:** La imagen del carácter recortado se redimensiona preservando su relación de aspecto mediante *letterbox resize* a 128×128 píxeles con relleno blanco, se convierte de BGR a RGB y se normaliza con la media y desviación estándar de ImageNet.
2. **Extracción de características:** El *backbone* EfficientNetV2-S procesa la imagen y produce un vector de 1 280 características mediante *global average pooling*.
3. **Proyección:** El *Projection Head* reduce el vector a 512 dimensiones, aplicando normalización por lotes, activación ReLU y regularización por *Dropout*.
4. **Clasificación:** Se calcula la similitud coseno entre el *embedding* normalizado (L2) y los 107 prototipos de clase (pesos de ArcFace, también normalizados), escalada por el factor $s = 30$. En inferencia no se aplica el margen angular m .
5. **Post-procesamiento:** Se aplica *softmax* a los logits para obtener probabilidades. Opcionalmente, se promedian las probabilidades de 5 variantes de TTA para mayor robustez ante variaciones de trazo.

2.3.3. Estrategia de entrenamiento

El entrenamiento del clasificador OCR se realizó en **Kaggle Notebooks** con GPU y se estructuró en tres fases progresivas, con un máximo de 50 épocas y un *timer* de seguridad de 110 minutos para respetar los límites de sesión de Kaggle:

1. **Fase de congelamiento** (5 épocas): Solo se entrenan el *Projection Head* y la cabeza ArcFace, manteniendo el *backbone* EfficientNetV2-S completamente congelado. Esta fase permite estabilizar los pesos de las nuevas capas antes de perturbar las características preentrenadas.
2. **Fase de calentamiento** (*warmup*, 3 épocas): Se descongela el *backbone* con un esquema discriminativo de tasas de aprendizaje: capas tempranas reciben 1×10^{-5} , intermedias 3×10^{-5} y tardías 1×10^{-4} . La tasa de aprendizaje global sube linealmente del 10 % al 100 % del valor objetivo durante esta fase.

3. **Fase completa** (hasta 42 épocas restantes): Todas las capas se entrenan con un *scheduler* `CosineAnnealingLR` sin reinicio ($T_{\text{máx}} = 45$, $\eta_{\text{mín}} = 10^{-6}$) y parada temprana (*early stopping*) de paciencia 12 épocas sobre la pérdida de validación. *Mixup* ($\alpha = 0.2$, garantizando $\lambda \geq 0.5$) se aplica en todas las épocas de esta fase.

2.3.4. Marco metodológico

Se utilizó una metodología **ágil (Scrum adaptado)**, realizando iteraciones cortas para el entrenamiento de los modelos y el desarrollo de los endpoints de la API. Esta elección se justifica por la naturaleza experimental de la IA: los resultados de cada iteración —análisis de matrices de confusión, identificación de pares confusos, ajuste de hiperparámetros— dictan los siguientes pasos del desarrollo. Las iteraciones del clasificador OCR comprendieron cinco versiones progresivas (v1 a v5), registradas mediante **MLflow**, con mejoras sucesivas en el manejo de caracteres acentuados, la partición honesta de datos y la estrategia de entrenamiento por fases.

2.3.5. Tendencias actuales

El proyecto se alinea con las siguientes tendencias en IA aplicada:

- **AI-as-a-Service (AlaaS):** Las funcionalidades de inteligencia artificial se exponen a través de APIs REST para que múltiples clientes (aplicaciones móviles, web, IoT) puedan consumirlas sin necesidad de hardware especializado en el dispositivo.
- **Angular Margin Learning:** El uso de ArcFace, originalmente desarrollado para reconocimiento facial, aplicado a clasificación de caracteres manuscritos refleja la creciente transferencia de técnicas entre dominios de visión artificial.
- **Exportación a formatos estándar:** La adopción de ONNX como formato de intercambio desacopla el entrenamiento (PyTorch en GPU) del despliegue (ONNX Runtime en CPU), facilitando la portabilidad y reduciendo costos de infraestructura.
- **Data-Centric AI:** El enfoque del proyecto en la calidad y composición de los datos —*Accent Augmentation*, partición honesta, muestreo ponderado por fuente—

refleja la tendencia actual de priorizar la ingeniería de datos sobre la complejidad del modelo para obtener ganancias reales de rendimiento.

2.3.6. Comparativa de tecnologías

Se eligió **FastAPI** sobre **Flask** o **Django** por su velocidad de ejecución asíncrona y la autogeneración de documentación Swagger, crítica para que futuros desarrolladores integren aplicaciones *front-end* sin fricción.

Para el modelo de clasificación de caracteres se eligió **EfficientNetV2-S** sobre arquitecturas más pesadas (ResNet-152, ViT-Large) por su equilibrio entre precisión y velocidad de inferencia en CPU, y sobre arquitecturas más ligeras (MobileNet) por su mayor capacidad para distinguir las 107 clases, incluyendo pares visualmente similares como caracteres acentuados. La cabeza **ArcFace** fue seleccionada sobre una capa *softmax* convencional tras verificar experimentalmente que mejora la separación de clases confusas (a/á, c/C, 1/1).

Para la detección se eligió **YOLOv8s** sobre modelos de detección de dos etapas (Faster R-CNN) por su capacidad de operar en un único paso de inferencia con latencias competitivas, y sobre versiones más grandes de YOLO (YOLOv8m, YOLOv8l) por el balance entre precisión y velocidad en la tarea concreta de detección de caracteres manuscritos en imágenes de 640×640 píxeles.

Capítulo 3

Metodología y desarrollo del proyecto

El desarrollo de **Tutor_API** se llevó a cabo bajo un esquema de trabajo híbrido. Durante las primeras semanas, la asistencia presencial fue fundamental para el levantamiento de requerimientos y la definición técnica con los asesores. Posteriormente, el desarrollo e implementación se realizaron de manera remota, manteniendo sesiones de revisión periódicas para asegurar la calidad del código y la precisión de los modelos de IA.

3.1. Levantamiento de requerimientos

Para comprender las necesidades del programa “**Chiapas Puede**”, se realizaron entrevistas iniciales presenciales con el Dr. Horacio Solís Cisneros. En estas sesiones se identificó que el sistema debía procesar fotografías de cuadernos escolares de forma rápida, ya que el usuario final —niños y adultos en proceso de alfabetización— requiere retroalimentación inmediata sobre la calidad de su escritura.

Se utilizó la técnica de **observación indirecta** de materiales didácticos de alfabetización para entender qué caracteres manuscritos debían ser reconocidos y evaluados, incluyendo los propios del español: vocales acentuadas (á, é, í, ó, ú, ü) y la ñe (ñ). Dado el carácter híbrido de la estadía, la validación de requerimientos se complementó mediante reuniones virtuales donde se definieron las restricciones técnicas: uso de Python, arquitectura asíncrona, inferencia sin GPU en el servidor de producción y soporte para fotografías tomadas con celular en condiciones variables de iluminación.

3.2. Análisis de requerimientos

A partir de la información recopilada, se clasificaron los requerimientos para estructurar el desarrollo de la API.

Requerimientos Funcionales (RF):

- **RF1:** La API debe recibir una fotografía de un carácter manuscrito individual, detectarlo con YOLOv8s, reconocerlo con el clasificador OCR (107 clases) y retornar una calificación de trazo (0–100) con retroalimentación pedagógica en español (endpoint `/evaluate`).
- **RF2:** La API debe procesar fotografías de planas escolares completas, detectar y evaluar todos los caracteres presentes, e informar la calificación promedio y el detalle por carácter (endpoint `/evaluate_plana`).
- **RF3:** La API debe reconocer y transcribir texto manuscrito a partir de una imagen, agrupando las predicciones individuales en palabras mediante SmartOCR con corrección por diccionario y distancia de Levenshtein (endpoint `/recognize`).
- **RF4:** El clasificador OCR debe soportar los 107 caracteres del español, incluyendo letras mayúsculas y minúsculas, dígitos, vocales acentuadas, eñe, dígrafos (ch, ll) y signos de puntuación.
- **RF5:** El sistema debe generar plantillas de referencia (`.npy`) para los tres niveles de dificultad —principiante (tolerancia 8 px), intermedio (5 px) y avanzado (3 px)— mediante el script `generate_templates.py`.
- **RF6:** El pipeline de procesamiento de imagen debe operar correctamente sobre fotografías reales de cuaderno con líneas azules, variaciones de iluminación e inclinación de hasta ± 15 .

Requerimientos No Funcionales (RNF):

- **RNF1 (Rendimiento):** Las peticiones de evaluación de carácter individual deben procesarse en menos de 500 ms en CPU y menos de 100 ms con GPU, sin cargar los modelos en cada petición.

- **RNF2 (Exactitud):** El clasificador OCR debe alcanzar al menos el 75 % de exactitud sobre datos manuscritos reales, medido sobre un conjunto de prueba compuesto exclusivamente por imágenes reales (no sintéticas).
- **RNF3 (Escalabilidad):** La arquitectura debe permitir agregar nuevas clases de caracteres o reemplazar modelos sin rediseñar el núcleo de la API.
- **RNF4 (Portabilidad):** Los modelos deben exportarse a formato ONNX para ejecutar inferencia sin PyTorch en el servidor de producción y facilitar el despliegue en contenedores Docker.
- **RNF5 (Usabilidad):** La documentación de la API debe generarse automáticamente en Swagger (/docs) para facilitar la integración con aplicaciones *front-end*.

3.3. Diseño

La arquitectura de **Tutor_API** se basa en un estilo de **capas con separación de responsabilidades**, donde el núcleo de IA, el pipeline de procesamiento de imagen y los endpoints de la API están claramente desacoplados.

- **Atributos de calidad:** Se priorizó la *mantenibilidad* mediante el uso de **Pydantic** para la validación de esquemas de entrada y salida, y la *disponibilidad* mediante la naturaleza asíncrona de **FastAPI** con **Uvicorn** como servidor ASGI. La *portabilidad* se garantiza exportando ambos modelos (detector y clasificador) a formato **ONNX**, lo que elimina la dependencia de PyTorch en producción.
- **Patrones de diseño:** Se implementó el patrón **Fachada** (*Facade*) en el módulo `processor.py` (v4.2), que actúa como orquestador del pipeline completo: recibe la imagen del endpoint, coordina la detección YOLO, el pipeline de limpieza de imagen (`image_cleaner.py`), la generación de la máscara binaria (`normalizer.py` v6), la clasificación OCR y el cálculo de las 8 métricas de trazo. Esta separación permite modificar cualquier componente interno sin alterar los contratos de la API.

- **Arquitectura de dos pipelines paralelos:** Un aspecto central del diseño es que la misma imagen del cuaderno alimenta dos pipelines independientes de forma simultánea:

- **Pipeline A (OCR):** Aplica limpieza por inpainting, convierte a escala de grises continua (256 niveles) y la envía al clasificador EfficientNetV2-S + ArcFace. Requiere los matices de gris para distinguir detalles finos entre clases similares.
- **Pipeline B (Métricas de trazo):** Aplica limpieza, binarización por percentiles y morfología para producir una máscara binaria 128×128 (blanco = trazo, negro = fondo). Esta máscara se esqueletiza y se compara con la plantilla de referencia mediante las 8 métricas especializadas.

Combinar ambos propósitos en un único pipeline produciría resultados inferiores en las dos tareas, razón por la cual se diseñaron como rutas independientes que convergen solo en la etapa de puntuación final.

- **Doble red de seguridad en procesamiento de imagen:** Para garantizar que el sistema genere siempre una máscara válida incluso ante fotografías de baja calidad, se diseñaron dos capas de fallback:

1. `normalizer.py v6` intenta tres estrategias en orden: pipeline principal con `image_cleaner`, pipeline *legacy* (HSV + Otsu) y pipeline de emergencia (desenfoque agresivo + Otsu).
2. `processor.py v4.2` valida la máscara resultante con cuatro heurísticas (% de píxeles activos, fragmentación, densidad de *hull convexo*) y, si la detecta como inválida, genera una máscara de emergencia directamente desde la imagen en escala de grises limpia.

Figura 3.1: Diagrama de componentes de Tutor_API: relación entre endpoints, orquestador, pipelines de imagen y módulos de métricas.

3.4. Implementación

La implementación se estructuró en tres frentes paralelos: el entrenamiento de los modelos en Kaggle, el desarrollo de la API en FastAPI y la construcción del pipeline de procesamiento de imagen.

3.4.1. Carga de modelos al iniciar el servidor

Un aspecto crítico fue garantizar que los modelos de IA —el detector YOLOv8s y el clasificador EfficientNetV2-S exportado a ONNX— se cargaran una única vez al iniciar el servidor y no en cada petición, evitando latencias inaceptables. FastAPI permite esto mediante el evento `lifespan` o variables globales inicializadas en el arranque:

```
1 # app/core/config.py (fragmento simplificado)
2 import onnxruntime as ort
3 from ultralytics import YOLO
4
5 # Los modelos se instancian una vez al importar el modulo
6 classifier_session = ort.InferenceSession(
7     "app/models/classifier_artifacts/best_classifier.onnx",
8     providers=["CUDAExecutionProvider", "CPUExecutionProvider"]
9 )
10
11 detector_model = YOLO(
12     "app/models/classifier_artifacts/best_detector.pt"
13 )
```

Listing 3.1: Carga única de modelos al iniciar el servidor FastAPI

3.4.2. Endpoint de evaluación de carácter individual

El endpoint `/evaluate` representa el flujo principal del sistema. Recibe la fotografía del cuaderno junto con el carácter esperado y el nivel de dificultad, orquesta los dos pipelines paralelos y devuelve la puntuación final con retroalimentación:

```
1 # app/api/endpoints.py (fragmento representativo)
```

```

2 from fastapi import APIRouter, UploadFile, File, Form
3 from app.core.processor import Processor
4
5 router = APIRouter()
6 processor = Processor() # instanciado una vez al cargar el modulo
7
8 @router.post("/evaluate")
9 async def evaluate(
10     file: UploadFile = File(...),
11     target_char: str = Form(...),
12     level: str = Form(default="intermedio")
13 ):
14     image_bytes = await file.read()
15     result = processor.evaluate(
16         image_bytes=image_bytes,
17         target_char=target_char,
18         level=level
19     )
20     return result

```

Listing 3.2: Endpoint principal de evaluación de trazo (/evaluate)

El objeto result que devuelve `processor.evaluate()` incluye el carácter detectado, la confianza del clasificador, la puntuación final (0–100), el desglose por las 8 métricas de trazo, la retroalimentación pedagógica y las imágenes de comparación codificadas en Base64.

3.4.3. Pipeline de limpieza de imagen

El módulo `image_cleaner.py` resuelve el problema de las líneas azules de los cuadernos de renglón, que interfieren con la binarización del trazo. El proceso detecta las líneas por su rango de color HSV (H: 85–130, S: 40–255, V: 50–255), dilata la máscara para cubrir bordes difusos y las elimina mediante la función de inpainting de OpenCV (`cv2.INPAINT_TELEA`), preservando únicamente el trazo del alumno:

```

1 # app/core/image_cleaner.py (fragmento)

```

```

2 import cv2
3 import numpy as np
4
5 def remove_ruled_lines(img_bgr: np.ndarray) -> np.ndarray:
6     hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
7     lower = np.array([85, 40, 50])
8     upper = np.array([130, 255, 255])
9     mask_lines = cv2.inRange(hsv, lower, upper)
10    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
11    mask_dilated = cv2.dilate(mask_lines, kernel, iterations=2)
12    cleaned = cv2.inpaint(
13        img_bgr, mask_dilated, inpaintRadius=3,
14        flags=cv2.INPAINT_TELEA
15    )
16    return cleaned

```

Listing 3.3: Limpieza de líneas de cuaderno por inpainting

3.4.4. Entrenamiento de modelos en Kaggle

Ambos modelos se entrenaron en **Kaggle Notebooks** con GPU T4 (16 GB VRAM) y se exportaron a ONNX para su integración en la API:

- **Detector YOLOv8s:** Entrenado durante 80 épocas con *batch size* 32 sobre un dataset compuesto de imágenes reales de cuaderno e imágenes sintéticas multi-carácter generadas por composición de crops individuales sobre fondos variados. Se deshabilitaron los flips horizontales y verticales (`flipplr=0`, `flipud=0`) para evitar que el modelo aprenda caracteres espejados o invertidos.
- **Clasificador OCR (EfficientNetV2-S + ArcFace):** Entrenado durante hasta 50 épocas con un esquema de tres fases: congelamiento del *backbone* (5 épocas), *warmup* con tasas de aprendizaje discriminativas (3 épocas) y entrenamiento completo con CosineAnnealingLR (hasta 42 épocas restantes, con parada temprana de paciencia 12). Se alcanzó un 81.26 % de exactitud en validación y un 79.34 % en el

conjunto de prueba compuesto exclusivamente por datos reales, la métrica honesta del sistema.

3.5. Pruebas

El proceso de pruebas se dividió en tres etapas:

1. **Pruebas de modelos (Kaggle):** Durante el entrenamiento, cada versión del clasificador OCR (v1 a v5) fue evaluada sobre el conjunto de prueba con *Test-Time Augmentation* (TTA, 5 transformaciones), generando matrices de confusión (107×107), reporte de F1 por clase y análisis de los 10 pares de clases más confundidos. Estas métricas, registradas en MLflow, guiaron los ajustes de hiperparámetros en cada iteración: desde la incorporación de *Accent Augmentation* hasta el ajuste del peso $1.5 \times$ para clases acentuadas en la función de pérdida.
2. **Pruebas de integración (Postman / script):** Una vez levantado el servidor con `uvicorn app.main:app`, se realizaron peticiones HTTP a los tres endpoints con imágenes de prueba reales, verificando:
 - Que `/evaluate` retorne un JSON con los campos `score_final`, `scores_breakdown`, `feedback` y las imágenes codificadas en Base64.
 - Que `/evaluate_plana` detecte todos los caracteres de la plana y retorne el campo `smart_ocr` con el texto reconocido.
 - Que `/recognize` transcriba correctamente texto manuscrito con corrección por diccionario.

El repositorio incluye el script `test_with_real_data.py` para reproducir estas pruebas con datos propios.

3. **Pruebas de robustez del pipeline de imagen:** Se diseñaron casos de prueba específicos para las condiciones reales del programa “Chiapas Puede”: fotografías tomadas con baja iluminación, cuadernos con líneas azules de renglón, imágenes

con inclinación de hasta ± 15 y trazos con lápiz de bajo contraste. Para cada caso se verificó que la doble red de seguridad del `processor.py` (v4.2) generara una máscara válida y que el `score_final` fuera consistente con la calidad visual del trazo.

3.6. Despliegue

El sistema se despliega mediante **Uvicorn** como servidor ASGI, levantando la aplicación FastAPI con todos los modelos cargados en memoria:

```
1 # Instalar dependencias
2 pip install -r requirements.txt
3
4 # Generar plantillas de referencia (solo la primera vez)
5 python -m app.scripts.generate_templates
6
7 # Modo desarrollo (con recarga automatica)
8 uvicorn app.main:app --reload
9
10 # Modo produccion
11 uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Listing 3.4: Comandos de despliegue del servidor

Para facilitar el despliegue reproducible en cualquier entorno, el repositorio incluye un **Dockerfile** que empaqueta el servidor, las dependencias y los modelos ONNX en un contenedor autocontenido. Los modelos entrenados (`best_detector.pt`, `best_classifier.onnx` y `best_classifier.onnx.data`) deben colocarse en `app/models/classifier__artifacts/` antes de construir la imagen:

```
1 # Construir la imagen
2 docker build -t tutor-api .
3
4 # Ejecutar el contenedor
5 docker run -p 8000:8000 tutor-api
```

Listing 3.5: Despliegue con Docker

Una vez en ejecución, la documentación interactiva de la API está disponible en <http://localhost:8000/docs>, donde se pueden probar los tres endpoints directamente desde el navegador sin necesidad de herramientas adicionales.

Capítulo 4

Resultados

En este capítulo se presentan los resultados obtenidos tras el proceso de desarrollo e implementación de **Tutor_API**. Los resultados se exponen de manera estructurada, alineándolos con los objetivos específicos planteados al inicio del proyecto y evidenciando el cumplimiento de los entregables mediante métricas verificables y productos funcionales.

4.1. Resultados mapeados a objetivos específicos

4.1.1. Objetivo 1 — Infraestructura de backend (FastAPI)

Se consolidó una API funcional y asíncrona con tres endpoints operativos, documentación interactiva Swagger disponible en /docs y servidor ASGI Uvicorn configurado tanto para modo desarrollo (recarga automática) como para modo producción. La Tabla 4.1 resume las capacidades de cada endpoint verificadas durante las pruebas de integración.

Las latencias medidas en CPU durante las pruebas se mantuvieron dentro del rango de 200–500 ms por carácter individual, cumpliendo el requerimiento RNF1. Con GPU, las latencias se redujeron a 50–100 ms.

4.1.2. Objetivo 2 — Detector de caracteres YOLOv8s

Se entrenó el modelo **YOLOv8s** como detector *single-class* (clase 0 = *trazo*) durante 80 épocas con *batch size* 32 en Kaggle (GPU T4, 16 GB VRAM). El dataset de entre-

Tabla 4.1: Endpoints implementados y verificados en Tutor_API.

Endpoint	Entrada	Salida verificada
POST /evaluate	Imagen JPG/PNG + target_char + level	score_final (0–100), desglose de 8 métricas, feedback en español, imágenes en Base64
POST /evaluate_plana	Imagen JPG/PNG de plana completa + level	avg_score, n_detected, smart_ocr con texto reconocido, detalle por carácter
POST /recognize	Imagen JPG/PNG con texto manuscrito	text transcrito, palabras con correction_method (Levenshtein), confianza global

namiento combinó imágenes reales del cuaderno con composites sintéticos multi-carácter generados por composición de crops individuales sobre fondos variados (blanco, ruidoso, degradado, cuadriculado, papel), con una partición 80/10/10 (entrenamiento/validación/-prueba).

La Tabla 4.2 presenta las métricas obtenidas en el conjunto de prueba con umbrales $\text{conf}=0.25$, $\text{iou}=0.5$.

Tabla 4.2: Métricas del detector YOLOv8s en el conjunto de prueba.

Métrica	Valor
mAP@0.50	—
mAP@0.50:0.95	—
Precisión	—
Recall	—

Los modelos exportados y verificados son:

- `best_detector.pt` — pesos PyTorch para inferencia con Ultralytics en la API.
- `best_detector.onnx` — modelo ONNX (opset 17, simplificado) para inferencia sin PyTorch.

La validación *end-to-end* sobre planas sintéticas (celda D-6 del notebook) confirmó que el detector ordena correctamente las detecciones en orden de lectura (de arriba a abajo, de izquierda a derecha) mediante agrupación por línea con tolerancia de 15 píxeles.

4.1.3. Objetivo 3 — Clasificador OCR (EfficientNetV2-S + Arc-Face)

Se entrenó el clasificador en cinco versiones iterativas (v1–v5), registradas con **ML-flow**. La versión final (v5, run_id: 20260414_174147) alcanzó los resultados que se presentan en la Tabla 4.3.

Tabla 4.3: Métricas del clasificador OCR v5 en el conjunto de prueba.

Métrica	Valor	Observación
Mejor exactitud en validación	81.26 %	Sobre imágenes reales y <i>Accent Aug.</i>
Exactitud global en prueba	80.97 %	Incluye clases sintéticas (inflado)
Exactitud real en prueba	79.34 %	Solo datos manuscritos reales. Predictor de producción.
Exactitud acentuadas (<i>aug.</i>)	85.12 %	14 clases con <i>Accent Augmentation</i>
Exactitud solo sintéticas	97.62 %	31 clases; valor inflado, no representativo
F1-Score ponderado	0.8093	Promedio ponderado por número de muestras

La Tabla 4.4 detalla la composición del dataset y la partición utilizada para el entrenamiento y la evaluación honesta.

Tabla 4.4: Composición del dataset del clasificador OCR.

Partición	Imágenes	Clases reales	Clases sintéticas
Entrenamiento	99,354	62 + 14	31
Validación	16,005	62 + 14	31
Prueba	16,005	62 + 14	31

Nota sobre la evaluación honesta: Los conjuntos de validación y prueba contienen exclusivamente imágenes de fuentes reales (EMNIST, verack) y de *Accent Augmentation*. Las imágenes puramente sintéticas se destinaron en su totalidad al entrenamiento. Por esta razón, la métrica relevante para estimar el rendimiento en producción es la **exactitud real en prueba (79.34 %)**, no la global (80.97 %).

Los artefactos exportados y verificados son:

- `best_classifier.onnx + best_classifier.onnx.data` — modelo completo

(EfficientNetV2-S + *Projection Head* + pesos ArcFace) en formato ONNX para inferencia con ONNX Runtime sin PyTorch.

- `char_map.json` — mapeo bidireccional índice ↔ carácter Unicode para las 107 clases.

4.1.4. Objetivo 4 — Pipeline de evaluación de trazo

Se implementaron 8 módulos de métricas independientes combinados mediante pesos fijos para producir la puntuación final (0–100). La Tabla 4.5 presenta los módulos, sus pesos y la interpretación pedagógica de cada uno.

Tabla 4.5: Módulos de evaluación de trazo, pesos y descripción.

Módulo	Peso	Descripción
<code>dt_precision</code>	0.30	Cercanía del esqueleto del alumno al trazo ideal (distance transform de la plantilla)
<code>dt_coverage</code>	0.20	Porcentaje de la plantilla cubierto dentro del radio de tolerancia por nivel
<code>topology</code>	0.20	Estructura topológica correcta: número de <i>loops</i> , <i>endpoints</i> y <i>junctions</i>
<code>ssim</code>	0.12	Similitud estructural perceptual (SSIM) entre máscara del alumno y plantilla
<code>procrustes</code>	0.10	Similitud de forma tras alineamiento óptimo (traslación, rotación, escala)
<code>hausdorff</code>	0.04	Distancia máxima entre contornos; penaliza desviaciones puntuales grandes
<code>trajectory</code>	0.02	Similitud de trayectoria espacial (DTW) entre esqueletos ordenados
<code>cosine</code>	0.02	Similitud angular entre segmentos del esqueleto

Las plantillas de referencia (`.npy`) se generan mediante el script `app/scripts/generate_templates.py` para los tres niveles de dificultad con las tolerancias de la Tabla 4.6.

Tabla 4.6: Tolerancias del módulo `dt_coverage` por nivel.

Nivel	Tolerancia (px)	Interpretación pedagógica
principiante	8.0	Acepta trazos aproximados; para primeras prácticas
intermedio	5.0	Balance entre forma y precisión
avanzado	3.0	Requiere alta precisión en el trazo

4.1.5. Objetivo 5 — Pipeline robusto de procesamiento de imagen

Se implementó un pipeline de preprocesamiento capaz de operar sobre fotografías reales de cuaderno tomadas con celular. La Tabla 4.7 resume los desafíos resueltos y las técnicas aplicadas.

Tabla 4.7: Desafíos del procesamiento de imagen y soluciones implementadas.

Desafío	Módulo	Solución
Líneas azules de renglón	<code>image_cleaner.py</code>	Detección HSV (H: 85–130) + inpainting <code>cv2.INPAINT_TELEA</code>
Iluminación variable / sombras	<code>illumination.py</code>	Normalización de iluminación adaptativa
Bajo contraste del lápiz	<code>normalizer.py</code> v6	CLAHE + binarización por percentiles ($\text{umbral} = \text{fg} + (\text{bg} - \text{fg}) \times 0.45$)
Textura del papel	<code>binarizer.py</code>	Morfología: apertura + cierre
Inclinación (± 15)	<code>normalizer.py</code> v6	Corrección de inclinación (<i>deskew</i>) automática
Máscara inválida	<code>processor.py</code> v4.2	4 heurísticas de validación + fallback de emergencia

4.1.6. Objetivo 6 — Cliente de escritorio Kivy

Se desarrolló un cliente de escritorio funcional con **Kivy 2.3.0** ubicado en el directorio `kivy_app/` del repositorio. La aplicación permite al alumno capturar una imagen desde su dispositivo, enviarla al servidor `localhost:8000` mediante una petición HTTP y visualizar el resultado de la evaluación —puntuación, retroalimentación y comparativa gráfica

trazo/plantilla— directamente en pantalla.

Figura 4.1: Prototipo funcional del cliente de escritorio Kivy conectado a Tutor_API.

4.2. Cumplimiento de entregables

La Tabla 4.8 resume la relación entre los entregables comprometidos en el Capítulo 1 y el estado al finalizar el proyecto.

Tabla 4.8: Evidencia de cumplimiento de entregables.

ID	Entregable	Resultado / Evidencia
1	Repositorio de código (Tutor_API)	Código fuente completo en GitHub con arquitectura FastAPI, 3 endpoints operativos, Dockerfile y cliente Kivy.
2	Documentación de endpoints (Swagger)	Disponible en /docs con esquemas Pydantic de solicitud y respuesta para los 3 endpoints.
3	Modelos entrenados y exportados a ONNX	best_detector.pt + best_detector.onnx (YOLOv8s) y best_classifier.onnx + best_classifier.onnx.data (EfficientNetV2-S + ArcFace, 107 clases, 79.34 % exactitud real).
4	Reporte final de estadía	Presente documento.

Los resultados demuestran que es posible integrar técnicas de aprendizaje profundo —detección single-class con YOLOv8, reconocimiento de 107 clases con *metric learning* y ArcFace, evaluación de trazo con 8 métricas especializadas— en un servicio web ligero capaz de operar sobre fotografías reales de cuadernos escolares sin GPU en el servidor de producción, proporcionando una base técnica sólida y reproducible para el programa de alfabetización “Chiapas Puede”.

Capítulo 5

Conclusiones y recomendaciones

5.1. Conclusiones

La culminación de **Tutor_API** representa un hito significativo en mi formación como ingeniero. Tras el desarrollo del proyecto en el marco del programa “Chiapas Puede”, se han extraído las siguientes conclusiones:

- **La calidad del dato es más importante que el algoritmo:** Más allá de la programación en Python, este proyecto me permitió comprender que el éxito de una Inteligencia Artificial no reside únicamente en la arquitectura del modelo, sino en la calidad y composición de los datos con los que se entrena. Gracias a la guía del Dr. Horacio Solís Cisneros, aprendí que la limpieza de datos es el paso más crítico. Esto quedó evidenciado en el clasificador OCR: la diferencia entre la exactitud global (80.97 %) y la exactitud real en prueba (79.34 %) es consecuencia directa de incluir clases sintéticas en el dataset, y solo una **partición honesta** —evaluando exclusivamente sobre datos manuscritos reales— permite estimar con honestidad el rendimiento que el sistema tendrá en producción.
- **Cerrando la brecha teoría-práctica:** Las bases proporcionadas por el Dr. Carlos Alberto Díaz Hernández en la materia de IA fueron el cimiento necesario para enfrentar retos reales. Aprendí a pasar de los conceptos abstractos a una implementación asíncrona con FastAPI que realmente puede ser consumida por aplicaciones cliente.

Uno de los aprendizajes más concretos fue el diseño de dos pipelines paralelos para la misma imagen: el **Pipeline A** para reconocimiento OCR (requiere escala de grises continua) y el **Pipeline B** para evaluación de trazo (requiere máscara binaria). La teoría indica separar responsabilidades; la práctica demostró que mezclarlos produce resultados inferiores en ambas tareas.

- **¿Qué se haría diferente?:** Si tuviera que repetir el proyecto, dedicaría más tiempo inicial a la recolección de imágenes reales de cuadernos en contexto chiapaneco, especialmente de caracteres acentuados (á, é, ñ), que representaron el mayor desafío técnico del proyecto al no existir datasets públicos con muestras manuscritas reales de estas clases. La técnica de *Accent Augmentation* desarrollada en este proyecto fue una solución efectiva, pero datos reales colectados en campo siempre producirán modelos más robustos. Asimismo, establecería desde el inicio el protocolo de evaluación honesta (conjuntos de prueba solo con datos reales) para que todas las versiones del modelo sean comparables bajo el mismo criterio.
- **Nuevos horizontes:** Este proyecto abre las puertas a futuras investigaciones en *EdTech* para zonas rurales. Un paso lógico sería complementar la evaluación visual del trazo con modelos de **procesamiento de lenguaje natural (NLP)** para que el tutor no solo evalúe la forma de los caracteres, sino que también pueda comprender y generar retroalimentación contextual más rica, adaptada al nivel y al progreso individual del alumno. A más largo plazo, una interfaz de voz permitiría que el sistema sea accesible para quienes aún no dominan la lectura, el caso de uso central del programa “Chiapas Puede”.

5.2. Recomendaciones

Para asegurar la evolución y el escalamiento de **Tutor_API**, se proponen las siguientes recomendaciones técnicas y operativas:

- **Recolección de datos reales en campo:** Se recomienda que el programa “Chiapas Puede” establezca un protocolo de recolección de muestras manuscritas direc-

tamente de sus usuarios en Chiapas, priorizando los caracteres con menor exactitud en el sistema actual: vocales acentuadas, ñe y dígrafos (ch, ll). Incorporar estas imágenes en el reentrenamiento del clasificador OCR mediante *fine-tuning* sobre el modelo ONNX existente reducirá el margen de error del $\approx 20\%$ actual en datos reales y adaptará el sistema a las caligrafías específicas de la región.

- **Migración hacia inferencia en el dispositivo (Edge Computing):** La exportación del clasificador a formato **ONNX** ya realizada en este proyecto sienta las bases para una futura migración hacia inferencia directamente en el dispositivo del alumno, eliminando la dependencia de conexión a internet, una restricción crítica en zonas rurales de Chiapas. Se recomienda evaluar **ONNX Runtime Mobile** para Android/iOS como el camino de menor fricción, ya que los pesos del modelo están en el formato correcto y no requieren conversión adicional.
- **Integración de CI/CD con el Dockerfile existente:** El repositorio ya incluye un **Dockerfile** que empaqueta el servidor, las dependencias y los modelos ONNX en un contenedor autocontenido. Se recomienda integrarlo con una pipeline de **CI/CD** (por ejemplo, GitHub Actions) para que cada nuevo modelo entrenado y exportado a ONNX se despliegue automáticamente sin interrumpir el servicio, permitiendo actualizaciones frecuentes conforme se acumulen nuevos datos reales de campo.
- **Ampliación del sistema de evaluación de trazo:** Las 8 métricas actuales evalúan la *forma estática* del trazo. Una extensión natural sería capturar el *orden de los trazos* (secuencia de puntos en el tiempo) para detectar errores de dirección — como trazar una “a” de derecha a izquierda— que la evaluación geométrica actual no puede identificar. Esto requeriría que el cliente Kivy transmitiera la trayectoria temporal del lápiz en lugar de solo la imagen final.
- **Supervisión docente como eje del sistema:** Mantener siempre el rol del tutor académico como supervisor del sistema. La tecnología debe ser una herramienta que potencie la labor del docente, nunca un reemplazo, especialmente en procesos tan sensibles como la alfabetización inicial. Se recomienda diseñar un panel de

monitoreo sencillo que permita al docente revisar las calificaciones históricas del grupo y detectar los caracteres que sistemáticamente presentan mayor dificultad, retroalimentando tanto al alumno como al proceso de enseñanza.

Capítulo 6

Anexos

6.1. Tarjeta de contacto de la empresa

- **Nombre de la empresa o institución:**
- **Nombre del empleador:** Dr. Horacio Solís Cisneros
- **Puesto en la empresa o institución:**
- **Dirección electrónica de correo:**
- **Número telefónico:**

6.2. Tarjeta de contacto del alumno

- **Nombre completo:** Daniel Peregrino Pérez
- **Dirección electrónica de correo (personal):**
- **Número telefónico:**
- **LinkedIn:**
- **GitHub:** <https://github.com/DanielPPerez>

6.3. Carta de presentación

imagenes/carta-presentacion.png

6.4. Carta de aceptación

imagenes/carta-acceptacion.png

6.5. Carta de terminación

`imagenes/carta-terminacion.png`

6.6. Evaluación laboral

imagenes/evaluacion-laboral.png

6.7. Evaluación docente

imagenes/evaluacion-docente.png

6.8. Repositorio y artefactos del proyecto

El código fuente completo del proyecto, incluyendo la API, el cliente Kivy, el Dockerfile y los scripts de generación de plantillas, está disponible en el repositorio público:

`https://github.com/DanielPPerez/Tutor\_API`

Los notebooks de entrenamiento de los modelos están disponibles en Kaggle:

`https://www.kaggle.com/code/danielperegrinoperez/detector-train`

`https:`

`/www.kaggle.com/code/danielperegrinoperez/clasificador-ocr-spanish`

La Tabla 6.1 lista los artefactos principales generados durante el proyecto, su ubicación en el repositorio y su propósito.

6.9. Glosario de abreviaturas

[leftmargin=3.5cm, style=nextline]

API *Application Programming Interface*. Interfaz de programación de aplicaciones.

ASGI *Asynchronous Server Gateway Interface*. Interfaz asíncrona entre servidor web y aplicación Python.

ArcFace *Additive Angular Margin Loss*. Función de pérdida con margen angular para aprendizaje métrico.

CLAHE *Contrast Limited Adaptive Histogram Equalization*. Ecuación adaptativa de histograma con límite de contraste.

CNN *Convolutional Neural Network*. Red neuronal convolucional.

DTW *Dynamic Time Warping*. Alineamiento temporal dinámico.

Tabla 6.1: Artefactos técnicos del proyecto Tutor_API.

Artefacto	Ubicación	Propósito
<code>best_detector.pt</code>	<code>app/models/classifier_artifacts/</code>	Pesos del detector YOLOv8s para uso con Ultralytics
<code>best_detector.onnx</code>	<code>app/models/classifier_artifacts/</code>	Detector exportado a ONNX (opset 17)
<code>best_classifier.onnx</code>	<code>app/models/classifier_artifacts/</code>	Clasificador OCR completo en ONNX (EfficientNetV2-S + Projection Head + ArcFace)
<code>best_classifier.onnx.data</code>	<code>app/models/classifier_artifacts/</code>	Datos externos del modelo ONNX (debe estar junto al <code>.onnx</code>)
<code>char_map.json</code>	<code>app/models/classifier_artifacts/</code>	Mapeo bidireccional índice ↔ carácter Unicode para las 107 clases
<code>generate_templates.py</code>	<code>app/scripts/</code>	Script de generación de plantillas <code>.npy</code> y esqueletos para los 3 niveles de dificultad
<code>test_with_real_data.py</code>	Raíz del repositorio	Script de pruebas de integración con imágenes reales
<code>Dockerfile</code>	Raíz del repositorio	Contenedor para despliegue reproducible del servidor
<code>requirements.txt</code>	Raíz del repositorio	Dependencias fijadas al 15/03/2026

EMNIST *Extended MNIST*. Dataset extendido de caracteres manuscritos con más de 800 000 muestras en 62 clases.

HTR *Handwritten Text Recognition*. Reconocimiento de texto manuscrito.

NLP *Natural Language Processing*. Procesamiento de lenguaje natural.

OCR *Optical Character Recognition*. Reconocimiento óptico de caracteres.

ONNX *Open Neural Network Exchange*. Formato estándar para representación e intercambio de modelos de aprendizaje profundo.

SSIM *Structural Similarity Index*. Índice de similitud estructural entre imágenes.

TTA *Test-Time Augmentation*. Aumentación en tiempo de prueba para mejorar la robustez de la predicción.

YOLO *You Only Look Once*. Familia de modelos de detección de objetos en tiempo real de un solo paso de inferencia.